UNITÀ 3 · SVILUPPARE SOFTWARE

Il ciclo di vita del software

Cascata, incrementi, spirale: modi diversi di andare dall'idea al prodotto. E poi i requisiti scritti bene e i test che scovano i bug prima degli utenti.

PER COMINCIARE

Organizzate la festa di fine anno della classe. Come procedete?

SONDAGGIO

- **A** Decido tutto all'inizio, poi eseguo il piano
- **B** Preparo una versione base e la miglioro un po' alla volta
- **C** Chiedo continuamente agli altri che cosa vogliono
- **D** Prima penso a che cosa può andare storto

RIPASSO

Metti in ordine le fasi del ciclo di vita

1. Studio di fattibilità: conviene farlo? Con quali costi?
2. Analisi dei requisiti: che cosa deve fare
3. Rilascio: installazione presso gli utenti
4. Codifica: scrittura del programma
5. Test: verifica e collaudo
6. Progettazione: come sarà fatto
7. Manutenzione: correzioni e nuove versioni

ANALISI DEI REQUISITI

Che cosa deve fare, e come deve farlo

Requisiti funzionali

Le **funzioni** del sistema: che cosa fa.

- Il docente prenota un laboratorio per una data e un'ora.
- Il docente annulla una sua prenotazione.
- Il sistema invia un'email di conferma.

Requisiti non funzionali

Le **qualità e i vincoli**: come lo fa.

- La pagina si apre in meno di 2 secondi in 4G.
- Funziona su Chrome, Firefox e Safari recenti.
- Solo gli utenti della scuola possono accedere.

METTITI ALLA PROVA

Quale di questi è un requisito non funzionale?

QUIZ

A Lo studente può vedere l'orario dei laboratori

B Il sistema invia un promemoria il giorno prima

C Il docente può esportare le prenotazioni in un file CSV

D Con 500 utenti collegati insieme, la risposta resta sotto i 3 secondi

REQUISITI SCRITTI BENE

Un buon requisito si può verificare

Vago

"L'app deve essere veloce e facile da usare."

Veloce quanto? Facile per chi? Nessun test può dire se è rispettato.

Verificabile

"Un nuovo utente prenota un laboratorio in meno di 1 minuto, senza istruzioni."

Si misura: 5 docenti, un cronometro, un risultato.



AL COMPUTER · 15 minuti

I requisiti di Prenota-lab

1. Apri Writer o Google Documenti e inserisci una tabella di 4 colonne: **ID**, **Requisito**, **Tipo**, **Come lo verifico**.
2. Scrivi 5 requisiti funzionali (ID da RF1 a RF5).
3. Scrivi 3 requisiti non funzionali (da RNF1 a RNF3), ognuno con un numero e un'unità di misura.
4. Per ogni requisito descrivi in una riga la prova che dimostra che è rispettato.
5. Salva come **requisiti-prenotalab.odt**.

Controllo: 8 righe; ogni RNF contiene un numero (secondi, utenti, minuti…); nessun requisito usa parole come "veloce", "bello", "facile" senza una misura.

IL MODELLO A CASCATA

Una fase alla volta, senza tornare indietro



Pregi

Semplice da pianificare, molta documentazione, adatto se i requisiti sono chiari e stabili.

Difetti

Il cliente vede il prodotto solo alla fine: un requisito sbagliato si scopre tardi e costa molto.

UNA SORPRESA STORICA

L'articolo della "cascata" la criticava

> **Credo in questo concetto, ma l'implementazione descritta sopra è rischiosa e invita al fallimento.**
>
> **Winston W. Royce**, ingegnere statunitense, "Managing the Development of Large Software Systems", 1970

Royce disegnò la sequenza di fasi e subito dopo propose di aggiungere **ritorni indietro**, prototipi e il coinvolgimento del cliente. Il nome "cascata" arrivò dopo.

MODELLI ITERATIVI E INCREMENTALI

Un pezzo alla volta, migliorando a ogni giro

Incrementale

Il prodotto cresce per **pezzi funzionanti**. Prima versione di Prenota-lab: solo prenotare. Poi annullare, poi le email.

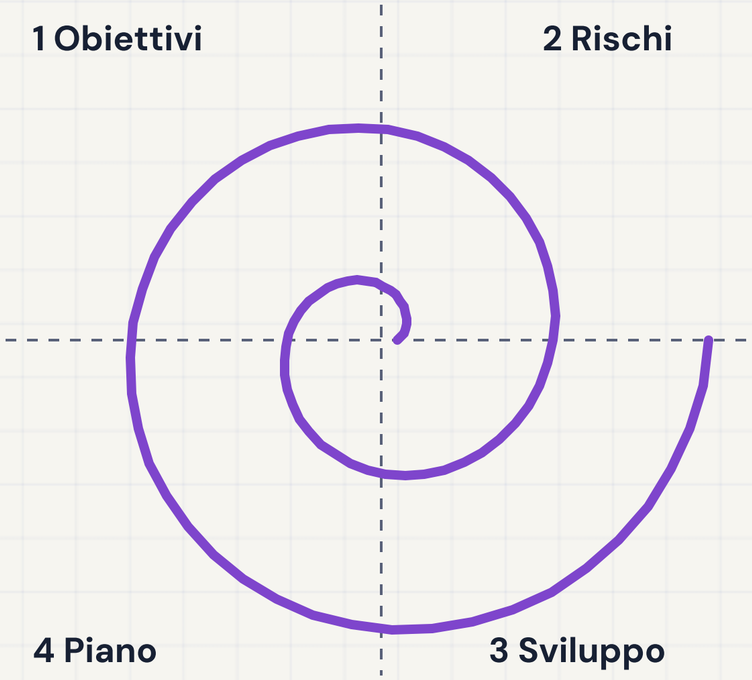
Iterativo

Si ripete il ciclo e si **raffina** ciò che c'è: prima un prototipo grezzo, poi sempre più rifinito grazie ai commenti degli utenti.

> Quasi sempre si usano insieme: ogni **iterazione** rilascia un **incremento**, cioè una versione più ricca che si può già usare.

IL MODELLO A SPIRALE

Ogni giro parte dai rischi



1. **Obiettivi**: che cosa vogliamo ottenere in questo giro.
2. **Analisi dei rischi**: che cosa può andare storto? Spesso si costruisce un prototipo per scoprirlo.
3. **Sviluppo e test** della nuova versione.
4. **Pianificazione** del giro successivo, con il cliente.

PUZZLE

Quale modello sceglieresti?

A cascata	Il cliente non sa ancora bene che cosa vuole
Incrementale	Progetto enorme e nuovo, con molti rischi tecnici
Iterativo con prototipi	Requisiti fissati da una legge, che non cambieranno
A spirale	Serve subito una prima versione usabile, il resto dopo

QUIZ A TRABOCCHETTO

A che cosa serve davvero il test?

QUIZ

- **A** A trovare errori, eseguendo il programma con casi scelti apposta
- **B** A dimostrare che il programma non ha errori
- **C** A controllare che il codice compili
- **D** A misurare quanto è veloce il programmatore

I LIVELLI DI TEST

Dal mattoncino al prodotto intero

1

Di unità

Un singolo metodo o classe, da solo.

2

Di integrazione

Più parti insieme: si parlano bene?

3

Di sistema

Tutto il prodotto, anche i requisiti non funzionali.

4

Di accettazione

Il cliente prova: è quello che voleva? Alfa e beta test.

Scatola nera

Scelgo i casi guardando solo i requisiti: ingressi e uscite attese.

Scatola bianca

Scelgo i casi guardando il codice: ogni ramo dell'`if` almeno una volta.

UN METODO E I SUOI TEST

Un anno è bisestile?

```
public class Main {
  static boolean bisestile(int anno) {
    return anno % 4 == 0;   // versione con un bug
  }
  static void prova(int anno, boolean atteso) {
    boolean ok = bisestile(anno) == atteso;
    System.out.println((ok ? "OK     " : "ERRORE ") + anno);
  }
  public static void main(String[] args) {
    prova(2024, true);
    prova(2023, false);   // aggiungi qui i tuoi test
  }
}
```

La regola vera: bisestile se divisibile per 4, ma gli anni secolari (1900, 2100) solo se divisibili per 400.

AL COMPUTER · 20 minuti

Scrivi i test che fanno cadere il bug

1. Apri **onlinegdb.com**, scegli Java e copia il programma della slide precedente.
2. Premi **Run**: i due test passano. Il metodo è giusto?
3. Aggiungi almeno 3 test con gli anni "difficili": 1900, 2000, 2100.
4. Leggi gli ERRORE, poi correggi il metodo `bisestile`.
5. Riesegui finché tutti i test stampano OK. Salva come **Bisestile.java**.

Controllo: con il metodo sbagliato compaiono `ERRORE 1900` e `ERRORE 2100`. Con il metodo corretto tutte le righe iniziano con OK.

PONTE SUL PRESENTE

Chi controlla i test scritti dall'AI?



Test ricavato dal codice

`prova(1900, true);` passa, ma il requisito dice il contrario: il bug resta nascosto.

Test ricavato dal requisito

`prova(1900, false);` fallisce e mostra il bug: è il test che serve.

AL COMPUTER · 20 minuti · a coppie

Prenota-lab in tre incrementi

1. Apri **excalidraw.com** (non serve un account).
2. Disegna 3 righe, una per incremento: in ognuna i riquadri Requisiti, Progetto, Codifica, Test e una freccia verso "Rilascio 1", "Rilascio 2", "Rilascio 3".
3. Accanto a ogni rilascio scrivi quali requisiti RF della tua tabella contiene.
4. Aggiungi una freccia di ritorno con scritto che cosa impari dagli utenti.
5. Esporta l'immagine come **incrementi-prenotalab.png**.

Controllo: il Rilascio 1 contiene già la funzione più importante (prenotare) e si può usare da solo; ogni requisito RF compare in un solo rilascio.

BIGLIETTO D'USCITA

Completa prima di uscire

"La pagina si apre in meno di 2 secondi" è un requisito _____.

Nel modello a _____ ogni fase finisce prima che inizi la successiva.

Il modello a spirale, di Barry Boehm, mette al centro l'analisi dei _____.

Il test che prova un singolo metodo da solo si chiama test di _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- Requisiti funzionali e non funzionali, sempre verificabili
- Cascata: semplice, ma il cliente vede tutto alla fine
- Iterativo e incrementale: versioni usabili e sempre migliori
- Spirale: ogni giro parte dall'analisi dei rischi
- Test di unità, integrazione, sistema, accettazione
- Un test trova i bug, non ne dimostra l'assenza

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.